Data Types & Type Systems

Liting Zhang

University of West Florida

Overview

- Primitive types
- Static vs dynamic typing
- Strong vs weak typing
- Type errors and type safety
- Introduction to the TypeScript type system
- Industry motivation for TypeScript
- Composite types: arrays, records, unions, enums
- Algebraic data types (conceptual)
- Generics and parametric polymorphism
- Subtyping and inheritance vs interfaces
- Nominal vs structural typing
- More TypeScript: interfaces, generics, unions, intersections
- Industry angle: designing API and library types

Primitive types across languages

Common primitive types:

- Numeric: int, float, double
- Boolean: true or false
- Character: single Unicode or byte char
- String: sequence of characters

How some languages handle them:

- C: int, double, char, no built-in string type (uses char arrays)
- Java, C#: int, double, boolean, char, String classes
- JavaScript: number, boolean, string (no separate int vs float)
- Python: int, float, bool, str
- Rust, Go: many numeric sizes (i32, u64, float32, etc.)

Numeric types and overflow

Fixed-size integers (C, C++, Java, C#, Rust):

- int32, int64, etc. have a maximum and minimum value.
- Overflow wraps around or is undefined, depending on language.

Unbounded integers (Python, Java BigInteger, Haskell Integer):

- Integers can grow arbitrarily large (limited by memory).
- No overflow, but arithmetic can be slower.

JavaScript and TypeScript:

- number is usually a 64-bit floating point value.
- BigInt exists for integers that cannot fit in number.

```c
// C: overflow wraps (on typical 32-bit int)
int x = 2147483647; // INT_MAX
x = x + 1;          // overflow: wraps to negative
```

```python
# Python: big integers, no overflow
x = 2 ** 100
print(x)  # very large integer
```

Primitive types: examples

```c
// C
int count = 10;
double price = 19.99;
char letter = 'A';
```

```python
# Python
count = 10              # int
price = 19.99           # float
letter = "A"            # string of length 1
flag = True             # bool
```

```javascript
// JavaScript
let count = 10;       // number
let price = 19.99;    // number
let name = "Alice";   // string
let done = false;     // boolean
```

Strings and characters across languages

Characters:

- C: char is usually one byte, often ASCII.
- Java, C#: char is a Unicode code unit.
- Python, JavaScript: no separate char type, just strings of length 1.

Strings:

- C: char arrays with '\0' terminator.
- Java, C#: immutable String objects.
- Python: immutable str, full Unicode.
- JavaScript, TypeScript: immutable string.

```c
// C string: char array with terminator
char msg[] = "hi";      // 'h', 'i', '\0'
```

```python
# Python string
msg = "hi"
ch = msg[0]    # "h"
```

Static vs dynamic typing

Static typing:

- Variable types are known at compile time.
- Types are checked before the program runs.
- Examples: C, C++, Java, C#, Rust, Go, TypeScript.

Dynamic typing:

- Values have types, variables can hold any type.
- Types are checked at runtime.
- Examples: Python, Ruby, JavaScript, Lua, PHP.

Mixed world:

- TypeScript adds static types on top of JavaScript.
- Many languages add optional type hints or type checkers.

Static vs dynamic: examples

```c
// C: static typing
int n = 10;
n = "hello";    // compile-time error
```

```python
# Python: dynamic typing
n = 10
n = "hello"     # allowed, type changes at runtime
```

```typescript
// TypeScript: static typing
let n: number = 10;
n = "hello";    // type error from TypeScript compiler
```

Static vs dynamic: trade-offs

Static typing (C, C++, Java, C#, Rust, Go, TypeScript):

- Catch many errors at compile time.
- Better IDE support and refactoring tools.
- More explicit design: types document intent.
- Sometimes more boilerplate, slower prototyping.

Dynamic typing (Python, Ruby, JavaScript, Lua, PHP):

- Very flexible and concise.
- Good for quick scripts and experiments.
- Type errors show up only when code path runs.
- Large codebases can become harder to maintain.

Many teams mix both:

- TypeScript + JavaScript.
- Python with optional type hints and type checkers.

Strong vs weak typing

Strong typing (informal idea):

- The language tries to prevent mixing incompatible types.
- Fewer implicit conversions between unrelated types.
- Examples: Python, Java, C#, Rust, Haskell.

Weak typing (informal idea):

- Language allows more implicit conversions.
- May convert between number and string automatically.
- C allows many casts, JavaScript coerces types in comparisons.

Note:

- Terms strong and weak are not precisely defined.
- You will see them used in slightly different ways.

Strong vs weak vs static vs dynamic

Two mostly independent axes:

- When are types checked? (static vs dynamic)
- How many implicit conversions are allowed? (strong vs weak)

Rough examples:

- Static and relatively strong: Java, C#, Rust, Haskell.
- Static and weaker: C (casts, pointer tricks).
- Dynamic and strong-ish: Python, Ruby.
- Dynamic and weaker: JavaScript with coercions.

Takeaway:

- Static vs dynamic is about timing of checks.
- Strong vs weak is about how strict the checks are.

Weak typing example: JavaScript

```
1 // JavaScript implicit conversions
2 1 + "2"          // "12"   (number + string -> string)
3 "3" * "4"        // 12     (strings converted to numbers)
4 true + 1         // 2
5 false == 0       // true
6 "0" == 0         // true
```

Python would behave differently:

```
1 1 + "2"      # TypeError in Python
2 "3" * "4"    # TypeError
3 True + 1     # result is 2, but bool is a subclass of int
```

Type errors and type safety

Type error:

- Program tries to use a value in a way that does not match its type.
- Examples: add integer to string, call something that is not a function.

Type safety:

- A language is type-safe if well-typed programs do not cause certain type errors at runtime.
- Static type systems try to catch type errors before execution.
- Dynamic checks catch type errors when that code path runs.

Examples:

- C is not type-safe: pointer casts can break abstract types.
- Java, C#, Rust, Haskell aim for stronger type safety.
- TypeScript increases type safety on top of JavaScript.

Type errors in practice

Compile-time type error (C / TypeScript):

```c
// C: compile-time error
int n = 10;
n = "hello";    // incompatible types
```

Runtime type error (Python):

```python
# Python: runtime error
x = 10
y = "20"
z = x + y  # TypeError happens when this line runs
```

Undefined behavior (C):

```c
// C: undefined behavior, not a well-defined type error
int *p = NULL;
*p = 42;    // may crash or do anything
```

What is TypeScript?

- Superset of JavaScript with static typing.
- Compiles down to plain JavaScript.
- Designed for large applications and teams.
- Adds types, interfaces, generics, modules, and more.
- Can be gradually added to existing JavaScript code.

Big picture:

- Same runtime as JavaScript.
- Better tooling: autocompletion, refactoring, safe renaming.
- Helps catch bugs before shipping to users.

TypeScript: basic annotations

```typescript
// variable annotations
let count: number = 0;
let name: string = "Alice";
let done: boolean = false;

// function annotations
function add(a: number, b: number): number {
  return a + b;
}

// arrays
let nums: number[] = [1, 2, 3];
let names: Array<string> = ["a", "b"];
```

TypeScript checks that you use these values in a type-safe way.

Type inference in TypeScript

TypeScript often infers types without explicit annotations.

```
1 let x = 10;           // inferred as number
2 let message = "hi"; // inferred as string
3
4 function square(n: number) {
5   return n * n;      // return type is inferred as number
6 }
7
8 let result = square(5); // result is number
9 result = "oops";         // error
```

Benefits:

- Less typing for the programmer.
- Still get the benefits of static type checking.

any vs unknown in TypeScript

any:

- Opt-out of type checking.
- You can do anything with a value of type any.

```
let value: any = 42;
value = "hello";
value.toUpperCase();    // allowed, no error
value.nonExisting();    // still allowed, no error
```

unknown:

- Safer alternative to any.
- You must check or narrow the type before using it.

```
let value: unknown = 42;
if (typeof value === "number") {
  let double = value * 2;     // ok, value is number here
}
// value.toUpperCase();   // error, unknown is not string
```

Why industry adopts TypeScript

Common reasons teams move from plain JavaScript to TypeScript:

- Better editor support and refactoring tools.
- Fewer runtime type errors in production.
- Clear contracts between modules and services.
- Easier onboarding for new team members.
- Works well with modern frameworks (React, Angular, Vue).

Other typed languages in industry:

- Backend: Java, C#, Go, Rust.
- Data and scripting: Python, R, Julia, Ruby.
- Systems: C, C++, Rust.

Composite types: arrays and records

Arrays and lists:

- Ordered sequence of elements.
- Usually all elements have the same type.

Records and objects:

- Group different fields under one name.
- Fields usually have different types.

Examples:

- C: arrays, structs.
- Java, C#: arrays, classes.
- Python: list, dict, namedtuple, dataclass.
- JavaScript, TypeScript: arrays, plain objects.

Arrays and records: examples

```c
// C
int nums[3] = {1, 2, 3};
struct Point {
  int x;
  int y;
};
struct Point p = { .x = 10, .y = 20 };
```

```typescript
// TypeScript
let nums: number[] = [1, 2, 3];
type Point = {
  x: number;
  y: number;
};
let p: Point = { x: 10, y: 20 };
```

Maps and dictionaries

Associative collections:

- Store key-value pairs.
- Often called maps, dictionaries, hash tables.

Common forms:

- C++: map, unordered_map.
- Java, C#: Map, HashMap, Dictionary.
- Python: dict.
- JavaScript: plain objects and Map.
- TypeScript: Record types and Map.

```python
# Python dict
ages = {"Alice": 30, "Bob": 25}
print(ages["Alice"])   # 30
```

```typescript
// TypeScript Record
type Ages = Record<string, number>;
const ages: Ages = { Alice: 30, Bob: 25 };
```

Unions and enums

Unions:

- A value that may be one of several types or variants.
- In C: union type (unsafe, manual use).
- In TypeScript: safe tagged unions with union types.

Enums:

- Fixed set of named constants.
- Often used for states or modes.

```c
// C union: unsafe if misused
union IntOrFloat {
  int i;
  float f;
};
int main(void) {
  union IntOrFloat x;
  x.i = 42;             // store an int
  // Now read as float: type punning, undefined behavior
  printf("%f\n", x.f);
  return 0;
}
```

Tagged unions pattern in C

Safer pattern in C:

- Combine an enum tag with a union.
- The tag says which union member is valid.

```
1  enum Kind { IS_INT, IS_FLOAT };
2
3  struct Value {
4    enum Kind tag;
5    union {
6      int i;
7      float f;
8    } u;
9  };
10
11 void print_value(struct Value v) {
12   if (v.tag == IS_INT) {
13     printf("int: %d\n", v.u.i);
14   } else {
15     printf("float: %f\n", v.u.f);
16   }
17 }
```

Unions and enums in typescript

```typescript
// TypeScript union and enum
enum Direction {
  North,
  South,
  East,
  West
}
type Result =
  | { kind: "ok"; value: number }
  | { kind: "error"; message: string };
```

Algebraic data types (informal idea)

Algebraic data types (ADTs):

- Built from simpler types using sum and product.
- Product type: combine fields (like records).
- Sum type: choose one of several variants (like unions).

Haskell style:

```haskell
data Result = Ok Int
            | Error String

data Point = Point Int Int
```

TypeScript can mimic this:

```typescript
type Result =
  | { kind: "ok"; value: number }
  | { kind: "error"; message: string };
```

Generics and parametric polymorphism

Parametric polymorphism:

- Functions and types that work for any type parameter.
- Often called generics in industry languages.

Examples:

- Java: List$< T >$, Map$< K, V >$.
- C#: List$< T >$, Dictionary$< TKey, TValue >$.
- C++: templates.
- Rust: Vec$< T >$, Option$< T >$.
- Haskell: list type [a], Maybe a.
- TypeScript: Array$< T >$, Promise$< T >$.

Polymorphism

Polymorphism = "many forms". Common kinds:
Ad-hoc polymorphism:

- Function overloading, operator overloading.
- Example: C++ operator+, Java method overloading.

Parametric polymorphism:

- Same code works for any type parameter.
- Example: Java List$< T >$, TypeScript Array$< T >$, Haskell [a].

Subtype polymorphism:

- Code works for a base type, can be called with subtypes.
- Example: Java Shape s = new Circle(); s.area().

Many modern languages support more than one kind at the same time.

Generics in TypeScript

```ts
function first<T>(items: T[]): T | undefined {
  return items[0];
}

let nums = [1, 2, 3];
let names = ["a", "b", "c"];

let n = first(nums);    // n is number | undefined
let s = first(names);   // s is string | undefined
```

Generic interface:

```ts
interface Box<T> {
  value: T;
}

let intBox: Box<number> = { value: 42 };
let strBox: Box<string> = { value: "hello" };
```

Subtyping and inheritance

Subtyping:

- If S is a subtype of T, values of S can be used where T is expected.

Inheritance:

- Class B extends class A and inherits fields and methods.
- Often introduces a subtype relation: B is a subtype of A.

Examples:

- Java: class Circle extends Shape.
- C#: class SqlConnection extends DbConnection.
- Many object oriented languages link inheritance and subtyping.

Nominal vs structural typing

Nominal typing:

- Types are equal only if they have the same name.
- Java, C#, C, C++ follow this approach for classes and structs.

Structural typing:

- Types are compatible if their structures match.
- TypeScript and Go use structural typing for many types.

Simple TypeScript example:

```
1  interface Point {
2    x: number;
3    y: number;
4  }
5  interface Pixel {
6    x: number;
7    y: number;
8  }
9  let p: Point = { x: 1, y: 2 };
10 let px: Pixel = p;  // ok: structure matches
```

Interfaces and structural typing in TypeScript

```typescript
interface Person {
  name: string;
  age: number;
}
function greet(p: Person) {
  console.log("Hello, " + p.name);
}
const user = {
  name: "Alice",
  age: 30,
  email: "a@example.com"
};
greet(user); // ok: user has at least name and age
```

Only the needed fields and types must match.

Union and intersection types in TypeScript

Union types:

```
type Id = number | string;
function printId(id: Id) {
  if (typeof id === "string") {
    console.log(id.toUpperCase());
  } else {
    console.log(id.toFixed(2));
  }
}
```

Intersection types:

```
type HasId = { id: number };
type HasName = { name: string };
type User = HasId & HasName;
const u: User = { id: 1, name: "Bob" };
```

Designing robust API types

When designing a library or service API:

- Use interfaces and type aliases to describe data shapes.
- Use enums or string unions for finite choices.
- Use generics to keep APIs reusable.
- Avoid any in public APIs when possible.
- Use union types or result types for errors and success.

Example result type in TypeScript:

```
type ApiResult<T> =
  | { ok: true; data: T }
  | { ok: false; error: string };
```

This pattern also appears in Rust (Result$< T, E >$) and Haskell (Either e a).